

Backend Júnior com Golang

Do zero ao projeto profissional com Go, PostgreSQL, Docker, APIs, testes e cloud

Trilha prática para primeira vaga



E-book de estudos e projeto de portfólio
Versão 1.0 - 2026

Página editorial

Título: Backend Júnior com Golang: Do zero ao projeto profissional com Go, PostgreSQL, Docker, APIs, testes e cloud.

Objetivo: servir como material de estudo para alunos iniciantes ou em transição para desenvolvimento backend, com foco em empregabilidade e construção de portfólio.

Licença sugerida: Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International para o e-book. Para o código do repositório, a licença MIT é uma boa opção.

Aviso: este material é educacional. Ferramentas, versões e serviços de cloud mudam com o tempo; revise dependências e configurações antes de usar em produção.

Como citar: Backend Júnior com Golang, versão 1.0, 2026.

Sumário

Página editorial	2
Sumário	3
1. Como usar este e-book	6
Ciclo de estudo recomendado	6
Resultado esperado	6
2. Mapa geral do backend	6
Analogia do restaurante	6
O que faz um desenvolvedor backend	7
Checkpoint	7
3. O que é Go e por que usar Golang	7
Características importantes	7
Primeiro programa	8
4. Preparando o ambiente	8
Comandos iniciais	8
Exercício	8
5. Fundamentos da programação com Go	8
Variáveis	8
Tipos básicos	9
Constantes	9
6. Controle de fluxo: if, else, switch e for	9
Para que serve if?	9
else e else if	9
Operadores	9
switch	10
for	10
Exercícios	10
7. Funções, pacotes e módulos	10
Função com erro	10
Pacotes	10
8. Arrays, slices e maps	11
Arrays e slices	11
Maps	11
Exercícios	11

9. Structs, métodos e interfaces	11
Struct	11
Métodos	11
Interfaces	11
10. Ponteiros em Go	12
11. Erros em Go	12
Erros personalizados	12
Mapeando erros para HTTP	12
12. HTTP, APIs, REST e JSON	13
Métodos HTTP	13
Status codes	13
Servidor HTTP simples	13
13. PostgreSQL e SQL	14
CRUD em SQL	14
Relacionamentos do projeto	14
Chave primária, estrangeira e índice	14
14. Migrations	14
Exemplo up/down	15
15. Arquitetura backend em camadas	15
Responsabilidades	15
Estrutura recomendada	15
16. Autenticação com JWT	16
bcrypt	16
Endpoints	16
Regras básicas	16
17. Docker e Docker Compose	16
Dockerfile	16
docker-compose.yml	17
Comandos úteis	17
18. Testes unitários e de integração	17
Teste unitário	18
Teste de integração	18
O que testar	18
19. Swagger e OpenAPI	18

Por que usar	18
20. Redis, cache e filas	18
TTL	19
Filas	19
21. Logs, observabilidade e qualidade	19
Graceful shutdown	19
22. Deploy em cloud	19
Opções recomendadas	20
Variáveis de ambiente	20
23. Projeto final de portfólio	20
Project Management API	20
Funcionalidades obrigatórias	20
Entidades principais	20
Endpoints recomendados	21
Regras de negócio	21
24. Plano de estudos de 12 semanas	21
25. Glossário essencial	22
26. Checklist final para candidatura	22
27. Roteiro de entrevista técnica	23
Perguntas comuns	23
Como explicar seu projeto	23
Conclusão	23

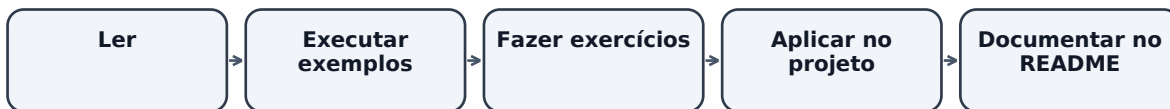
1. Como usar este e-book

Este e-book foi organizado como um curso prático. A proposta não é apenas ler, mas implementar. Cada capítulo apresenta conceitos, exemplos em Go, exercícios e checkpoints de compreensão.

O projeto central da formação é uma API chamada Project Management API. Ao final, ela deverá conter usuários, autenticação, projetos, tarefas, membros, comentários, banco PostgreSQL, Docker, testes, documentação e deploy.

Ciclo de estudo recomendado

Ciclo de aprendizagem ativa



Sempre que estudar uma tecnologia, conecte-a ao projeto. Isso transforma teoria em evidência técnica para entrevistas e portfólio.

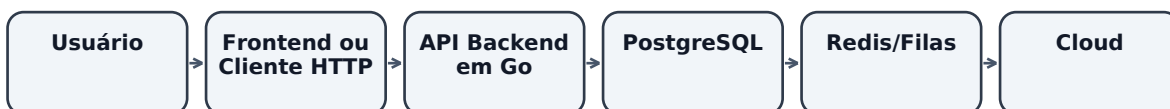
Resultado esperado

- Criar APIs REST em Go.
- Conectar Go ao PostgreSQL.
- Implementar autenticação JWT.
- Rodar a aplicação com Docker Compose.
- Escrever testes unitários e de integração.
- Documentar endpoints com Swagger/OpenAPI.
- Usar Redis para cache simples.
- Fazer deploy básico em cloud.
- Explicar decisões técnicas em entrevistas.

2. Mapa geral do backend

Backend é a parte da aplicação responsável por regras de negócio, dados, segurança, autenticação, integrações e processamento. O usuário normalmente não interage diretamente com o backend; ele usa uma tela, aplicativo ou cliente HTTP.

Visão geral de uma aplicação backend



Analogia do restaurante

Elemento no restaurante	Elemento no sistema
-------------------------	---------------------

Cliente	Usuário
Garçom	Frontend
Cozinha	Backend
Estoque	Banco de dados
Comanda	Requisição HTTP
Prato entregue	Resposta HTTP

O que faz um desenvolvedor backend

- Cria APIs e endpoints.
- Modela tabelas e consultas SQL.
- Implementa regras de negócio.
- Cuida de autenticação e autorização.
- Integra serviços externos.
- Escreve testes automatizados.
- Publica aplicações em cloud.
- Analisa logs e falhas.

Checkpoint

- Explique com suas palavras a diferença entre frontend e backend.
- Dê três exemplos de APIs usadas no dia a dia.
- Desenhe o fluxo de login de um sistema.

3. O que é Go e por que usar Golang

Go é uma linguagem criada pelo Google, conhecida por simplicidade, compilação rápida, boa performance e excelente uso em servidores, APIs, cloud, DevOps e sistemas distribuídos. O nome oficial é Go; Golang é um apelido popular.

Características importantes

Característica	Explicação
Compilada	O código é transformado em um executável.
Tipada estaticamente	O tipo das variáveis é conhecido antes da execução.
Concorrente	Possui goroutines e channels para executar tarefas simultâneas.
Simples	Tem sintaxe pequena e direta.
Boa para backend	Facilita APIs, serviços e ferramentas de infraestrutura.

Fluxo de compilação



Primeiro programa

```
package main

import "fmt"

func main() {
    fmt.Println("Olá, mundo!")
}
```

O pacote main indica que o programa pode ser executado. A função main é o ponto de entrada. O pacote fmt permite imprimir texto no terminal.

4. Preparando o ambiente

Antes de codar, prepare um ambiente reproduzível. Para primeira vaga, o avaliador técnico precisa conseguir clonar seu projeto e rodá-lo com instruções claras.

Ferramenta	Para que serve
Go	Compilar e executar código Go.
VS Code	Editor de código.
Git	Controle de versão.
GitHub	Portfólio e hospedagem do repositório.
Docker	Rodar aplicação em containers.
Docker Compose	Orquestrar API, banco e Redis localmente.
Postman/Insomnia/Bruno	Testar endpoints HTTP.

Comandos iniciais

```
go version
git --version
docker version
docker compose version

git init
git add .
git commit -m "initial commit"
```

Exercício

Crie um repositório chamado go-backend-studies, adicione um arquivo main.go e publique no GitHub com um README simples.

5. Fundamentos da programação com Go

Variáveis

Variável é um espaço de memória que guarda um valor. Em backend, variáveis representam dados como e-mail, senha, ID, status, preço, token e parâmetros de requisição.

```
var nome string = "Ana"
var idade int = 25
email := "ana@email.com"
ativo := true
```


Tipos básicos

Tipo	Uso
string	Textos.
int	Números inteiros.
float64	Números decimais.
bool	Verdadeiro ou falso.

Constantes

```
const AppName = "Project Management API"
const MaxPageSize = 100
```

Use constantes para valores que não mudam durante a execução.

6. Controle de fluxo: if, else, switch e for

Para que serve if?

O if serve para tomar decisões. Ele executa um bloco somente quando uma condição é verdadeira.

```
idade := 18
```

```
if idade >= 18 {
    fmt.Println("Maior de idade")
}
```

Fluxo de decisão com if



else e else if

```
nota := 7
```

```
if nota >= 9 {
    fmt.Println("Excelente")
} else if nota >= 7 {
    fmt.Println("Aprovado")
} else {
    fmt.Println("Reprovado")
}
```

Operadores

Operador	Significado
==	Igual
!=	Diferente
>	Maior que
<	Menor que

>=	Maior ou igual
<=	Menor ou igual
&&	E lógico
	OU lógico
!	Negação

switch

O switch escolhe entre várias opções de forma mais limpa que muitos if/else.

```
status := "pending"

switch status {
case "pending":
    fmt.Println("Pendente")
case "done":
    fmt.Println("Concluído")
default:
    fmt.Println("Status desconhecido")
}
```

for

O for serve para repetir instruções. Go possui apenas um comando de repetição: for.

```
for i := 0; i < 5; i++ {
    fmt.Println(i)
}
```

Exercícios

- Verifique se uma pessoa pode votar.
- Imprima os números de 1 a 100.
- Use switch para imprimir mensagens por status de tarefa.
- Some os números de 1 a 10.

7. Funções, pacotes e módulos

Função é um bloco de código reutilizável. Uma boa função deve ter uma responsabilidade clara e um nome que explique sua intenção.

```
func somar(a int, b int) int {
    return a + b
}
```

Função com erro

```
func dividir(a float64, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("divisão por zero")
    }
    return a / b, nil
}
```

Pacotes

Pacote é uma forma de organizar arquivos Go relacionados. Em um backend profissional, você normalmente separa handlers, services, repositories, models, middlewares e configurações.

```
go mod init github.com/seuusuario/project-management-api
```

8. Arrays, slices e maps

Arrays e slices

Array possui tamanho fixo. Slice é uma coleção dinâmica e é muito mais usado em Go para representar listas.

```
nomes := []string{"Ana", "Bruno", "Carlos"}
nomes = append(nomes, "Daniela")
```

```
for indice, nome := range nomes {
    fmt.Println(indice, nome)
}
```

Maps

Map guarda pares de chave e valor. Em backend, é útil para representar associações rápidas, configurações e estruturas similares a dicionários.

```
idades := map[string]int{
    "Ana": 25,
    "Bruno": 30,
}
fmt.Println(idades["Ana"])
```

Exercícios

- Crie um slice com cinco tarefas.
- Crie um map de produto e preço.
- Crie uma função que recebe notas e retorna média.

9. Structs, métodos e interfaces

Struct

Struct agrupa dados relacionados e representa entidades do domínio.

```
type User struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Email string `json:"email"`
}
```

Métodos

```
type Task struct {
    Title string
    Status string
}

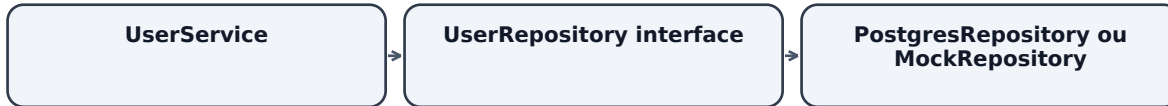
func (t Task) IsDone() bool {
    return t.Status == "done"
}
```

Interfaces

Interface define um contrato de comportamento. Ela permite que o service dependa de uma abstração em vez de uma implementação concreta.

```
type UserRepository interface {  
    Create(user User) error  
    FindByEmail(email string) (User, error)  
}
```

Interface reduz acoplamento e facilita testes



10. Ponteiros em Go

Ponteiro guarda o endereço de memória de outra variável. Ele é usado quando queremos alterar o valor original ou evitar cópias desnecessárias.

```
idade := 30  
ponteiro := &idade  
fmt.Println(*ponteiro) // valor apontado  
  
func renomear(nome *string) {  
    *nome = "Novo nome"  
}
```

Em métodos, use ponteiro quando o método altera o estado original da struct.

```
func (t *Task) MarkAsDone() {  
    t.Status = "done"  
}
```

11. Erros em Go

Em Go, erros são valores. A forma idiomática é retornar um erro e verificar se ele é diferente de nil.

```
result, err := dividir(10, 0)  
if err != nil {  
    fmt.Println("erro:", err)  
    return  
}  
fmt.Println(result)
```

Erros personalizados

```
var ErrUserNotFound = errors.New("user not found")  
  
if errors.Is(err, ErrUserNotFound) {  
    // retornar HTTP 404  
}
```

Mapeando erros para HTTP

Erro de negócio	Status HTTP
Usuário não encontrado	404 Not Found
Credenciais inválidas	401 Unauthorized
Sem permissão	403 Forbidden
Entrada inválida	400 Bad Request
Erro inesperado	500 Internal Server Error

12. HTTP, APIs, REST e JSON

HTTP é o protocolo usado na web. APIs REST organizam recursos por URLs e usam métodos HTTP para representar ações.

Fluxo HTTP básico



Métodos HTTP

Método	Uso
GET	Buscar dados.
POST	Criar dados.
PUT	Substituir dados.
PATCH	Atualizar parcialmente.
DELETE	Remover dados.

Status codes

Status	Significado
200 OK	Sucesso.
201 Created	Criado com sucesso.
400 Bad Request	Requisição inválida.
401 Unauthorized	Não autenticado.
403 Forbidden	Sem permissão.
404 Not Found	Não encontrado.
500 Internal Server Error	Erro interno.

Servidor HTTP simples

```
package main

import (
    "encoding/json"
    "net/http"
)

type HealthResponse struct {
    Status string `json:"status"`
}

func healthHandler(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusOK)
```

```
    json.NewEncoder(w).Encode(HealthResponse{Status: "ok"})
}

func main() {
    http.HandleFunc("/health", healthHandler)
    http.ListenAndServe(":8080", nil)
}
```

13. PostgreSQL e SQL

PostgreSQL é um banco relacional robusto e muito usado no mercado. SQL é a linguagem usada para criar, consultar e alterar dados em bancos relacionais.

CRUD em SQL

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

INSERT INTO users (name, email, password_hash)
VALUES ('Ana', 'ana@email.com', 'hash');

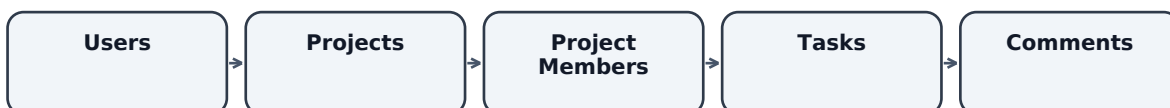
SELECT id, name, email FROM users;

UPDATE users SET name = 'Ana Silva' WHERE id = 1;

DELETE FROM users WHERE id = 1;
```

Relacionamentos do projeto

Modelo relacional simplificado



Chave primária, estrangeira e índice

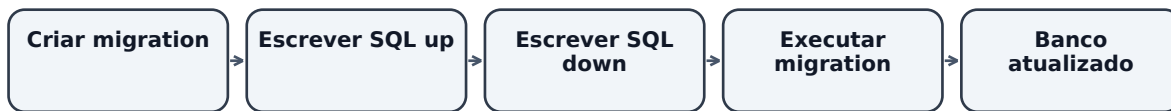
Termo	Para que serve
Primary key	Identifica um registro de forma única.
Foreign key	Liga uma tabela a outra.
Index	Acelera consultas em campos frequentes.

14. Migrations

Migrations são arquivos que versionam mudanças no banco de dados. Elas evitam alterações manuais desorganizadas e permitem reproduzir o banco em qualquer ambiente.

```
001_create_users_table.up.sql
001_create_users_table.down.sql
002_create_projects_table.up.sql
002_create_projects_table.down.sql
```

Fluxo de migrations



Exemplo up/down

```
-- up
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL
);
```

```
-- down
DROP TABLE users;
```

15. Arquitetura backend em camadas

Organizar o backend em camadas deixa o código mais testável, legível e próximo de projetos profissionais.

Arquitetura em camadas



Responsabilidades

Camada	Responsabilidade
Handler	Ler request, chamar service e retornar response.
Service	Concentrar regras de negócio e permissões.
Repository	Executar SQL e acessar o banco.
Model	Representar entidades e DTOs.
Middleware	Interceptar requisições para autenticação, logs etc.
Config	Carregar variáveis de ambiente.

Estrutura recomendada

```
project-api/
├── cmd/api/main.go
├── internal/config/
├── internal/handler/
├── internal/service/
├── internal/repository/
├── internal/model/
├── internal/middleware/
├── migrations/
└── docs/
```

```
├── Dockerfile
├── docker-compose.yml
├── .env.example
├── go.mod
└── README.md
```

16. Autenticação com JWT

Autenticação verifica quem é o usuário. Autorização verifica o que ele pode fazer. JWT é um token usado para representar a identidade do usuário após login.

Fluxo de autenticação JWT



bcrypt

Bcrypt gera hash de senha. Nunca salve senha pura no banco, pois um vazamento exporia todas as credenciais.

Authorization: Bearer token_aqui

Endpoints

POST /auth/register
POST /auth/login
GET /me

Regras básicas

- Usuário cria conta com nome, email e senha.
- Senha é salva como hash.
- Login valida email e senha.
- API retorna JWT.
- Rotas protegidas exigem Authorization: Bearer token.

17. Docker e Docker Compose

Docker empacota aplicações em containers. Docker Compose permite subir vários containers juntos, como API, PostgreSQL e Redis.

Ambiente local containerizado



Dockerfile


```
FROM golang:1.22 AS builder
WORKDIR /app
COPY . .
RUN go build -o api ./cmd/api

FROM debian:bookworm-slim
WORKDIR /app
COPY --from=builder /app/api .
CMD ["/api"]
```

docker-compose.yml

```
services:
  api:
    build: .
    ports:
      - "8080:8080"
    depends_on:
      - postgres
    env_file:
      - .env

  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: app
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7
    ports:
      - "6379:6379"

volumes:
  postgres_data:
```

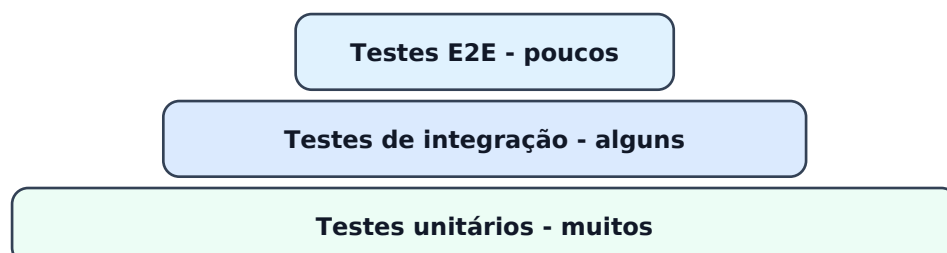
Comandos úteis

```
docker compose up --build
docker compose down
docker compose logs -f api
docker ps
```

18. Testes unitários e de integração

Testes automatizados verificam se o código funciona e protegem contra regressões. Para portfólio, testes demonstram maturidade acima da média para júnior.

Pirâmide de testes



Teste unitário

```
func TestSomar(t *testing.T) {
    resultado := somar(2, 3)
    if resultado != 5 {
        t.Errorf("esperado 5, obtido %d", resultado)
    }
}
```

Teste de integração

Teste de integração verifica várias partes juntas, por exemplo handler, service, repository, migrations e PostgreSQL.

O que testar

- Cadastro de usuário.
- Login.
- Criação de projeto.
- Criação de tarefa.
- Permissões.
- Token ausente ou inválido.
- Validação de campos.

19. Swagger e OpenAPI

OpenAPI é uma especificação para documentar APIs. Swagger é um conjunto de ferramentas para visualizar e testar essa documentação.

Por que usar

- Facilita testes manuais.
- Ajuda frontend a integrar.
- Mostra profissionalismo no GitHub.
- Permite que recrutador técnico entenda a API rapidamente.

```
paths:
  /health:
    get:
      summary: Healthcheck da aplicação
      responses:
        '200':
          description: Aplicação saudável
```

20. Redis, cache e filas

Redis é um banco em memória, rápido, usado para cache, sessões, rate limit e filas simples. Para primeira vaga, cache com Redis já é um ótimo diferencial.

Fluxo de cache com Redis



TTL

TTL significa Time To Live. É o tempo de vida de um dado em cache. Exemplo: guardar uma listagem por 60 segundos.

Filas

Fila permite processar tarefas depois, de forma assíncrona. RabbitMQ é comum para filas tradicionais; Kafka é usado para streaming de eventos em grande escala.

21. Logs, observabilidade e qualidade

Logs registram eventos da aplicação. Observabilidade é a capacidade de entender o que ocorre no sistema por meio de logs, métricas e traces.

Conceito	Uso
Logs	Registrar eventos e erros.
Métricas	Medir números como latência e taxa de erro.
Traces	Acompanhar o caminho de uma requisição.
Healthcheck	Indicar que a aplicação está viva.
Readiness	Indicar que a aplicação está pronta para receber tráfego.
Lint	Encontrar problemas e más práticas no código.

Graceful shutdown

Graceful shutdown é desligar a aplicação com cuidado: parar de aceitar novas requisições, terminar requisições em andamento, fechar conexões e encerrar o processo.

22. Deploy em cloud

Deploy é publicar a aplicação para que outras pessoas possam acessá-la pela internet. Para portfólio, ter uma API online aumenta muito a credibilidade do projeto.

Fluxo de deploy



Opções recomendadas

Opção	Perfil
Render/Railway	Mais simples para portfólio.
Fly.io	Bom para apps pequenos.
VPS com Docker	Mais profissional e manual.
AWS Lightsail	Boa entrada no ecossistema AWS.

Variáveis de ambiente

```
DATABASE_URL=postgres://user:pass@host:5432/db
JWT_SECRET=segredo
APP_ENV=production
```

Nunca publique segredos reais no GitHub.

23. Projeto final de portfólio

Project Management API

API REST desenvolvida em Go para gerenciamento de projetos, tarefas e membros, com autenticação JWT, PostgreSQL, Docker, testes, Redis, Swagger/OpenAPI e deploy em cloud.

Funcionalidades obrigatórias

- Cadastro de usuários.
- Login com JWT.
- CRUD de projetos.
- CRUD de tarefas.
- Membros por projeto.
- Permissões básicas.
- Paginação e filtros.
- Validação de dados.
- Tratamento padronizado de erros.
- Testes unitários e de integração.
- Swagger/OpenAPI.
- Docker Compose.
- Deploy público.

Entidades principais

Entidade	Campos principais
User	id, name, email, password_hash, created_at
Project	id, name, description, owner_id, created_at, updated_at
ProjectMember	project_id, user_id, role, created_at
Task	id, project_id, title, description, status, assigned_to, created_at, updated_at

Comment	id, task_id, user_id, content, created_at
---------	---

Endpoints recomendados

POST /auth/register

POST /auth/login

GET /me

POST /projects

GET /projects

GET /projects/{id}

PUT /projects/{id}

DELETE /projects/{id}

POST /projects/{id}/members

GET /projects/{id}/members

DELETE /projects/{id}/members/{user_id}

POST /projects/{id}/tasks

GET /projects/{id}/tasks

GET /tasks/{id}

PUT /tasks/{id}

DELETE /tasks/{id}

POST /tasks/{id}/comments

GET /tasks/{id}/comments

Regras de negócio

- Usuário precisa estar autenticado para criar projeto.
- Criador do projeto vira owner.
- Somente membros acessam o projeto.
- Somente owner adiciona ou remove membros.
- Tarefa pertence a um projeto.
- Somente membros podem criar tarefas.
- Usuário não pode acessar projeto do qual não é membro.

24. Plano de estudos de 12 semanas

Semana	Foco	Entrega
1	Go básico para backend	Programas pequenos, /health e repositório.
2	HTTP e JSON	API simples com GET e POST.
3	PostgreSQL e SQL	API conectada ao banco.
4	Migrations e arquitetura	Projeto em camadas com migrations.
5	CRUD completo	Projetos e tarefas com validação.
6	JWT e autenticação	Register, login, /me e rotas protegidas.
7	Docker	API, PostgreSQL e Redis via Compose.
8	Testes unitários	Services testados.
9	Testes de integração	Fluxos críticos com banco real/teste.
10	Swagger e documentação	OpenAPI e README profissional.
11	Redis ou fila	Cache ou mensageria funcionando.
12	Deploy e candidatura	API online, currículo e LinkedIn atualizados.

25. Glossário essencial

Termo	Significado
API	Interface para sistemas conversarem.
REST	Estilo de arquitetura baseado em recursos e métodos HTTP.
HTTP	Protocolo de comunicação da web.
JSON	Formato textual para troca de dados.
SQL	Linguagem para bancos relacionais.
JWT	Token usado para autenticação stateless.
CRUD	Create, Read, Update, Delete.
ORM	Mapeamento entre banco relacional e objetos/structs.
CLI	Interface de linha de comando.
SDK	Kit de desenvolvimento para uma plataforma.
VPS	Servidor virtual privado.
CI/CD	Integração e entrega/deploy contínuos.
TTL	Tempo de vida de um dado em cache.
SRE	Engenharia de confiabilidade de sites/sistemas.
DevOps	Práticas que aproximam desenvolvimento e operações.
Endpoint	URL específica de uma API.
Middleware	Código intermediário entre request e handler.
Handler	Função que recebe request e retorna response.
Service	Camada de regras de negócio.
Repository	Camada de acesso a dados.
Migration	Versionamento de alterações no banco.
Deploy	Publicação da aplicação.
Hash	Transformação segura de dados, usada para senhas.

26. Checklist final para candidatura

- Sei explicar backend, API, HTTP, REST e JSON.
- Sei criar endpoints em Go.
- Sei usar if, else, for, switch, funções, structs, interfaces e erros.
- Sei conectar Go com PostgreSQL.
- Sei escrever SQL básico e migrations.
- Sei organizar projeto em camadas.
- Sei implementar JWT e bcrypt.
- Sei rodar tudo com Docker Compose.
- Sei escrever testes unitários e integração básicos.

- Sei documentar com Swagger/OpenAPI.
- Sei usar Redis ou explicar cache.
- Tenho deploy público.
- Tenho README profissional.
- Sei explicar decisões técnicas.

27. Roteiro de entrevista técnica

Perguntas comuns

Pergunta	Resposta esperada
O que é API?	Interface que permite comunicação entre sistemas por contratos como endpoints, métodos e respostas.
Diferença entre 401 e 403?	401: não autenticado. 403: autenticado, mas sem permissão.
O que é JWT?	Token usado para representar identidade após login, enviado no header Authorization.
Por que não salvar senha pura?	Porque vazamento do banco exporia credenciais. Use hash com bcrypt.
O que é Docker?	Tecnologia para empacotar aplicações em containers.
O que são migrations?	Arquivos que versionam mudanças no banco.
Unitário vs integração?	Unitário testa parte isolada; integração testa partes trabalhando juntas.

Como explicar seu projeto

Meu projeto é uma API de gerenciamento de projetos e tarefas. Ela permite cadastro de usuários, login com JWT, criação de projetos, membros, tarefas e comentários. Organizei o código em camadas: handlers para HTTP, services para regra de negócio e repositories para banco. Usei PostgreSQL com migrations, Docker Compose para ambiente local, testes automatizados, Swagger para documentação, Redis para cache e deploy em cloud.

Conclusão

A primeira vaga em backend exige fundamentos, prática e capacidade de explicar o que foi construído. Este e-book guia o aluno do básico ao avançado prático usando Go como linguagem principal e um projeto completo como eixo central.

A melhor forma de aprender backend é construir backend, explicar suas decisões e melhorar continuamente o projeto. Ao final, você deverá ter um projeto backend completo, um README profissional, uma API online, base para entrevistas técnicas, vocabulário técnico e segurança para se candidatar a vagas júnior.